PRODUCT REQUIREMENTS DOCUMENT

SkillPass BD

Digital Professional Identity & Verified Work History Platform for Bangladesh's Informal Skilled Workforce

Version	1.2 (draft)
Date	2026-10-25
Status	Draft — for team review
Tech stack	Django · Django REST Framework · React (Vite) · PostgreSQL
Source	SkillPass BD Project Proposal

University Software Engineering Project

Revision History

Version	Date	Changes
1.0	2026-10-24	Initial draft from project proposal
1.1	2026-10-25	Low-literacy worker experience: PIN login, assisted onboarding, call/map buttons, new-job sound alert, voice help, PWA, Help screen (FR-01, FR-03, FR-10, FR-17, FR-22, §14, §15); release plan changed to part-by-part delivery (§17)
1.2	2026-10-25	Tech stack pinned to installed versions (React 19, Vite 8, Tailwind 4, React Router 7, Django 5.2 LTS, PostgreSQL 16+); local PostgreSQL instead of Docker; routes centralized in `app/router.jsx` (§16)

Table of Contents

1. Overview
2. Problem Statement
3. Goals and Non-Goals
4. Users and Personas
5. Core Concepts and Glossary
6. User Journeys
7. Functional Requirements
8. Booking / Job State Machine
9. Trust Score Model
10. Matching Algorithm
11. Roles and Permissions
12. Data Model
13. API Specification
14. UI / UX Requirements
15. Non-Functional Requirements
16. Technical Architecture
17. Release Plan and Milestones
18. Success Metrics
19. Risks and Mitigations

20. Open Questions

1. Overview

SkillPass BD gives informal skilled workers in Bangladesh — electricians, plumbers, AC technicians, appliance repair technicians, mechanics, painters, cleaners — a **digital "skill passport"**: a portable professional profile backed by verified identity, structured skills, customer-confirmed work history, structured reviews, and an explainable trust score.

Customers use the same platform to find, compare, and book workers based on evidence rather than word of mouth.

What makes it different from a normal service marketplace: the product's core asset is the worker's *long-term professional identity*, not the individual booking. Every confirmed job adds permanent, verifiable evidence to the worker's profile.

1.1 Priority Legend

Priority	Meaning
P0	MVP. Required for the core demo. Must ship.
P1	Important. Ship after P0 is stable.
P2	Optional / stretch. Ship if time allows.

2. Problem Statement

For workers:

- No standardized digital professional identity.
- Past work is scattered across phone contacts and Facebook pages and cannot be shown to new customers.
- Strong practical skill does not turn into a portable reputation.

For customers:

- No reliable way to evaluate an unfamiliar worker before hiring.
- Reviews elsewhere are unstructured (a single star rating) and say little about reliability.
- No visibility into completion rate, cancellations, response behavior, or verified history.

Gap in existing platforms: they optimize for booking a service, not for building a worker's long-term, verifiable reputation.

3. Goals and Non-Goals

3.1 Goals

#	Goal
G1	Workers can build a structured professional profile (skills, experience, service areas, availability).
G2	Customers can discover and compare workers using transparent, evidence-based information.
G3	A complete request → match → booking → completion → review workflow works end to end.
G4	Every customer-confirmed job becomes a permanent verified work-history record.
G5	Trust is shown as an explainable score with a visible breakdown, not a black box.
G6	Identity verification, skill evidence, and work history are verified separately.
G7	A worker with limited reading ability and little smartphone experience can accept and complete a job using pictures, big buttons, and Bengali voice help, with almost no typing.
G8	The codebase demonstrates solid software engineering: auth, RBAC, workflows, tests, API docs.

3.2 Non-Goals (Initial Release)

- Native Android / iOS apps (the web app is responsive; PWA is a future option).
- Automated NID verification against government systems.
- Online payments, escrow, or financial settlement.
- Real-time GPS tracking of workers.
- Large-scale commercial deployment.
- ML models that need large proprietary datasets.
- Real-time chat over WebSockets (MVP uses simple polling).

4. Users and Personas

4.1 Roles

Role	Description	Priority
Customer	Requests and books services, confirms completion, writes reviews.	P0
Worker	Maintains profile, accepts jobs, updates job status, builds verified history.	P0
Admin	Verifies workers, manages catalog, resolves disputes, moderates content.	P0
Contractor / Business	Manages a team of workers, assigns jobs, monitors performance.	P2

4.2 Personas

Rahim — AC Technician (Worker)

- 34, Mirpur, Dhaka. 9 years experience. Uses a budget Android phone on mobile data.
- Limited formal education: reads short Bengali text slowly, almost no English, avoids typing.
- Uses WhatsApp, phone calls, and YouTube; recognizes icons and pictures easily.
- Gets work through phone calls and word of mouth; prefers calling customers over chatting.
- *Needs:* show his experience to new customers, get more jobs, job actions he can do by tapping one obvious button, and someone or something to explain the app in spoken Bengali.

> **Design target:** most workers are like Rahim. Every worker screen is designed and tested against this persona first.

Nusrat — Homeowner (Customer)

- 29, Dhanmondi, office worker. Comfortable with apps.
- Moved recently; has no trusted technician nearby.
- *Needs:* find a reliable worker quickly, see proof of past work, track the job.

Karim — Platform Admin

- Reviews verification documents, handles complaints, maintains categories.
- *Needs:* efficient review queues, clear audit trail, dispute tools.

Shila — Small Cleaning Business Owner (Contractor, P2)

- Runs a team of 8 cleaners.
- *Needs:* assign jobs to team members, see team performance.

5. Core Concepts and Glossary

Term	Definition
SkillPass profile	The worker's public professional identity page.
Service category	Top-level service type (e.g., AC Repair, Plumbing).
Skill	Specific capability under a category (e.g., "AC gas refill", "Split AC installation").
Service area	Location hierarchy: Division → District → Area (thana / neighborhood).
Service request	A customer's description of a need: category, problem, area, preferred time.
Booking	A request assigned to one specific worker. A booking *is* the job and carries the job lifecycle.
Verified job	A booking that the customer explicitly confirmed as completed.
Self-reported work	Past work the worker adds manually (with optional photos). Always labeled "Not verified".
Identity verification	Confirms *who* the worker is (NID / photo check by admin).
Skill evidence	Documents supporting a *capability* (training certificate, trade license, photos).
Trust score	0–100 score calculated from measurable activity, with a visible breakdown.

Key rule: identity verification is never presented as proof of technical skill. The UI shows three separate badges: *Identity Verified*, *Skill Evidence Reviewed* (per skill), and *N Verified Jobs*.

> **Design decision:** The proposal lists `ServiceRequest`, `Booking`, and `Job` separately. This PRD merges `Booking` and `Job` into one entity (`Booking`) to avoid duplicated state. `ServiceRequest` stays separate because one request can be offered to a worker, declined, and re-offered to another.

6. User Journeys

6.1 Worker Onboarding

1. Worker registers with phone number + **6-digit PIN**, selecting role "Worker" by tapping a picture card. *Alternative (assisted onboarding):* an admin creates the account and gives the worker a temporary PIN, which the worker changes at first login.
2. Guided profile wizard (one question per screen, Bengali by default, 🔊 read-aloud on each step): name → photo (camera) → categories (picture tiles) → skills (picture tiles) → years of experience (number stepper) → service areas (tap list) → availability.
3. Worker optionally uploads identity documents and skill evidence by taking photos with the phone camera.
4. Profile goes live immediately with status "Unverified". Badges appear as admin approves items.
5. App suggests "Add to Home Screen" so SkillPass opens from an icon like a normal app.

6.2 Customer Finds and Books a Worker (Primary Flow)

1. Customer submits a service request: category, problem description, area, preferred date/time slot. *(P2: describe in free Bengali text; AI suggests category and tags.)*
2. System shows a ranked list of matched workers with trust score, verified jobs, rating, and distance-in-area.
3. Customer opens a worker's SkillPass profile and reviews history and breakdown.
4. Customer selects a worker → booking created with status `PENDING`.
5. Worker receives a notification and accepts or declines.
6. Worker progresses the job: **On the Way** → **Start** → **Mark Complete**.
7. Customer confirms completion → booking becomes `COMPLETED` and a verified work record is created.
8. Customer submits a structured review.
9. Worker's trust score recalculates.

6.3 Direct Booking from Profile

Customer browses or searches workers, opens a profile, and taps "Book". A service request is created implicitly, then the flow continues from step 4 above.

6.4 Dispute

1. Customer rejects the worker's completion claim, or either party reports a problem.
2. Booking moves to `DISPUTED`; admin is notified.
3. Admin reviews booking history and messages, then resolves as `COMPLETED` or `CANCELLED` with a note.

6.5 Admin Verification

1. Admin opens the verification queue (oldest first).
2. Admin views the document (served through an authenticated endpoint) and approves or rejects with a reason.
3. Worker is notified; badge appears on the profile; trust score recalculates.

7. Functional Requirements

Each requirement lists acceptance criteria (AC). IDs map to the proposal's FR-01..FR-15 where applicable.

FR-01 Authentication — P0

- Register with **phone number (BD format, unique)**, full name, role (Customer or Worker), and a credential. Email optional.
- **Customers and admins:** password (min 8 characters, Django password validators).
- **Workers: 6-digit numeric PIN**. A PIN is easier to remember and type than a password for workers with limited literacy.
- Login with phone + password/PIN (same endpoint and field). JWT access token (15 min) + refresh token (30 days for workers, 7 days for others) with rotation and blacklist on logout. Long refresh life keeps workers logged in so they rarely see the login screen.
- Change password/PIN (authenticated). Reset via email link if email exists (P1). Workers without email reset through admin/support, which issues a temporary PIN (P0). SMS OTP is P2.
- **Assisted onboarding (P1):** admin can create a worker account with a temporary PIN; the worker must set a new PIN at first login (`must_change_credential` flag).
- **AC:**
- Phone numbers are validated against `^(\+8801|01)[3-9]\d{8}$` and normalized to `+8801XXXXXXXXX`.
- Duplicate phone returns HTTP 400 with a field error.
- Worker PIN must be exactly 6 digits and not in a weak-PIN blocklist (`000000`, `123456`, `111111`, repeated or sequential digits, the last 6 digits of the phone number).
- The PIN is hashed with Django's password hasher, exactly like a password; it is never stored or logged in plain text.
- Logout blacklists the refresh token; reuse returns HTTP 401.
- Login is rate-limited (5 failed attempts / 15 min per phone + IP). Because a PIN has fewer combinations than a password, after 10 failed attempts in 24 h the account is locked until admin/support unlocks it.

FR-02 Role Management — P0

- Every user has exactly one role: `CUSTOMER`, `WORKER`, `CONTRACTOR`, or `ADMIN`.
- Admin accounts are created only via Django admin or a management command, never via public registration.
- Permissions enforced server-side via DRF permission classes and object-level checks (see §11).
- **AC:** A customer calling a worker-only endpoint gets HTTP 403. A user accessing another user's booking gets HTTP 404 (not 403, to avoid leaking existence).

FR-03 Worker Profile — P0

- Fields: display name, photo, bio (Bengali/English), years of experience, categories, skills, service areas, availability status, weekly availability slots.
- Availability status toggle: `AVAILABLE` / `BUSY` / `OFFLINE` — one tap from the worker dashboard.
- Public profile page shows: photo, name, badges, trust score + breakdown, skills (with evidence badge per skill), service areas, verified work history, self-reported work, reviews, member since.
- Profile completeness indicator (progress bar + next suggested step as a picture card).
- Categories and skills are chosen from **picture tiles** (icon + short Bengali label), never from a text-only list.

- Profile photo and documents are taken directly with the phone camera (`capture` attribute), with gallery as fallback.
- Bio is optional. Voice typing for the bio (Bengali speech-to-text where the browser supports it) is P1.
- **AC:** Phone number is never shown publicly; it is revealed to the customer only after a booking is `ACCEPTED`.

FR-04 Skill and Catalog Management — P0

- Admin manages categories and skills (name in Bengali + English, slug, icon, active flag).
- Workers attach skills from the catalog (no free-text skills in MVP) with an optional per-skill years of experience.
- Seed data: 6–8 categories, ~40 skills.
- **AC:** Deactivated skills disappear from search filters but remain visible on historical bookings.

FR-05 Service Areas and Availability — P0

- Location hierarchy: Division → District → Area. MVP seeds Dhaka division with Dhaka city areas (Mirpur, Dhanmondi, Uttara, Mohammadpur, Gulshan, Banani, Badda, etc.).
- Worker selects one or more areas they serve.
- Weekly availability: day of week + time slots (Morning / Afternoon / Evening) — P1.

FR-06 Service Request — P0

- Customer creates a request: category, optional skill(s), problem description, area, address detail, preferred date, preferred time slot, optional photos (max 3, P1).
- Request statuses: `OPEN` → `BOOKED` → `CLOSED`, or `CANCELLED`.
- **AC:** A request can have at most one active booking at a time. If a worker declines, the customer can book another worker from the same request.

FR-07 Matching — P0

- `GET /service-requests/{id}/matches/` returns ranked workers. Rule-based algorithm in §10.
- Each result includes the reasons it matched (e.g., "Serves Mirpur", "12 verified AC jobs").
- **AC:** Workers who are `OFFLINE`, suspended, or do not serve the area or category are never returned.

FR-08 Worker Discovery / Search — P0

- Public search: filter by category, skill, area, minimum trust score, identity-verified only, available now.
- Sort: best match (default), trust score, rating, verified jobs, experience.
- Paginated (20 per page).

FR-09 Booking — P0

- Customer creates a booking from a request + chosen worker.
- Worker must accept or decline within **24 hours**, else the booking auto-expires.
- Either party can cancel with a mandatory reason; who cancelled and at what state is recorded (affects trust).
- **AC:** A worker cannot hold two bookings in `IN_PROGRESS` at once.

FR-10 Job Tracking — P0

- Status transitions follow the state machine in §8. Invalid transitions return HTTP 409.
- Every transition writes a `BookingStatusHistory` row (from, to, actor, timestamp, note).

- Both parties see a status timeline.
- **Worker active job screen (P0):**
- Shows exactly **one** large primary button for the next step: গ্রহণ (Accept) → রওনা দিয়েছি (On the Way) → কাজ শুরু (Start) → কাজ শেষ (Complete). The button label, icon, and color change with the state.
- 📞 **Call customer** button (`tel:` link) once the booking is `ACCEPTED`.
- 🗺️ **Directions** button opening Google Maps with the booking address.
- Decline and Cancel are secondary, smaller, and always ask for confirmation with an icon-based dialog.
- **AC:** From the worker home screen, a worker can accept a new job in 1 tap and move it from accepted to completed in 3 more taps (plus confirmations only for destructive actions).

FR-11 Completion Verification — P0

- When the worker taps "Mark Complete", the booking moves to `AWAITING_CONFIRMATION`.
- Customer can **Confirm** (→ `COMPLETED`, counts as a verified job) or **Report a Problem** (→ `DISPUTED`).
- If the customer does nothing for **72 hours**, the booking auto-closes as `AUTO_CLOSED`: it counts toward completion rate but **not** toward verified jobs.
- Worker may attach completion notes and up to 3 photos (P1).

FR-12 Work History — P0 (verified) / P1 (self-reported)

- Verified entries are generated automatically from `COMPLETED` bookings: category, skills, area (area-level only, never full address), month/year, review summary.
- Self-reported entries (P1): worker adds title, category, description, approximate date, up to 3 photos. Always labeled "Self-reported — not verified". Excluded from trust score.
- **AC:** Workers cannot edit or delete verified entries. Customers' identities are shown only as first name + initial.

FR-13 Structured Reviews — P0

- Allowed only for the booking's customer, only when status is `COMPLETED` or `AUTO_CLOSED`, only once per booking, and only within 14 days.
- Fields: quality, professionalism, punctuality, communication (each 1–5), would-hire-again (yes/no), optional comment (max 1000 chars).
- Overall rating = mean of the four dimensions.
- Worker can post one public reply per review (P1).
- Admin can hide abusive reviews (hidden reviews still count in metrics unless admin marks them invalid).
- **AC:** Enforced by a DB unique constraint on `booking_id` and validation in the service layer.

FR-14 Verification — P0

- Verification types: `IDENTITY`, `SKILL_EVIDENCE` (linked to a specific worker skill).
- Worker uploads one or more documents (JPG/PNG/PDF, max 5 MB each).
- Statuses: `PENDING` → `APPROVED` / `REJECTED` (with reason). Worker can resubmit after rejection.
- Documents are stored privately and never publicly accessible. Only the owning worker and admins can fetch them, through an authenticated endpoint.
- **AC:** Every admin decision is written to the audit log.

FR-15 Trust Profile — P0

- Trust score 0–100 with a per-factor breakdown visible on the public profile (see §9).
- "How is this calculated?" explainer is always one tap away.
- Recalculated on: booking terminal state, review created/invalidated, verification decision, dispute resolution.
- History stored for a trend chart (P1).

FR-16 Reports and Disputes — P0

- Any party in a booking can raise a dispute on it. Any user can report a profile or review.
- Report reasons: no-show, poor work, overcharging, misconduct, fake profile, inappropriate content, other.
- Admin queue with statuses `OPEN` → `IN_REVIEW` → `RESOLVED` / `DISMISSED`, resolution note required.
- Admin actions: resolve booking outcome, hide review, warn user, suspend user.

FR-17 Notifications — P0

- In-app notifications for: new booking, booking accepted/declined/cancelled/expired, status changes, completion awaiting confirmation, review received, verification decision, dispute updates.
- Unread count badge; mark one / all as read.
- Delivery via frontend polling every 30 s in MVP. Email (P1), browser push / SMS (P2).
- **Worker new-job alert (P0):** when a new booking request arrives while the app is open, play a distinct sound, vibrate the phone (Vibration API where supported), and show a full-width green card at the top of the screen.
- Notification text for workers is short Bengali with an icon (e.g. "❄️ নতুন কাজ: AC সার্ভিস, মিরপুর").

FR-18 Messaging — P1

- Simple per-booking message thread between customer and worker (text only, max 1000 chars).
- Available from `PENDING` until 7 days after a terminal state, then read-only.
- Polling-based in MVP.

FR-19 Admin Dashboard — P0

- Overview metrics: users by role, active workers, bookings by status (last 30 days), pending verifications, open disputes.
- Queues: verification, disputes/reports.
- Management: users (search, suspend/reactivate), categories, skills, areas.
- Custom React admin pages for queues and metrics; Django admin remains available for raw data management.

FR-20 AI Assistance — P2 (Optional Module)

All AI features are **assistive and optional**. The core platform works fully without them, and every AI output is a suggestion the user confirms.

Feature	Input	Output
Bengali request understanding	"আমার AC ঠান্ডা করছে না এবং পানি পড়ছে"	Category: AC Repair; tags: not cooling, water leakage
Worker profile assistance	Worker's short Bengali description of experience	Suggested skills, years of experience, polished bio

Feature	Input	Output
Job summary	Request description + completion notes	Short plain-language summary on the work history entry

- Implemented behind an `AIProvider` interface in the backend (`ai` app) so the LLM vendor can be swapped.
- Fallback when AI is disabled or fails: keyword-based classifier using a Bengali/English keyword map per category.
- AI calls have a timeout (10 s) and never block the main workflow.

FR-21 Contractor / Business — P2

- Contractor creates an organization and invites existing workers by phone number.
- Contractor can view team bookings and performance and reassign a pending booking to a team member.
- Workers keep their own SkillPass profile and history even if they leave the organization (portability).

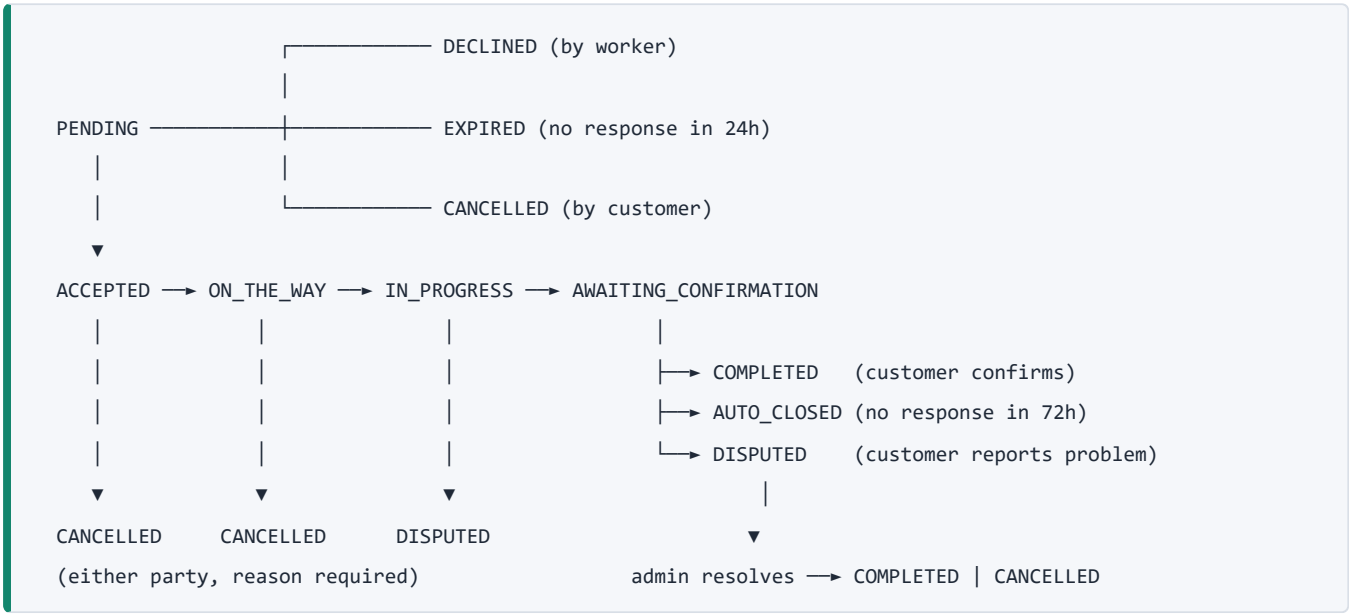
FR-22 Worker Help and Assistance — P0 / P1

Support for workers with limited literacy and little smartphone experience.

Feature	Priority	Description
Help screen	P0	Bottom-bar "Help" tab with a big 📞 **Call support** button and a **WhatsApp support** button (`tel:` / `wa.me` links), plus picture-based answers to the 5 most common questions.
Plain-language errors	P0	Every error shown to a worker is short Bengali that says what to do next (e.g. "নম্বরটি ভুল, আবার দিন"). No technical messages or codes.
Read-aloud (🔊)	P1	A speaker button on each worker screen reads the screen's instruction aloud in Bengali, using the browser's speech synthesis (`bn-BD`) when available, otherwise a short pre-recorded audio clip.
Voice typing	P1	Microphone button on free-text fields (bio, notes) using browser speech recognition (`bn-BD`) where supported.
Installable app (PWA)	P1	Web app manifest + service worker so the worker can "Add to Home Screen" and open SkillPass full-screen from an icon. Offline screen shows "ইন্টারনেট নেই" instead of a browser error.
Tutorial video	P1	A 1–2 minute Bengali screen-recording on the Help screen: register, accept a job, complete a job.
Assisted onboarding	P1	Admin creates a worker account with a temporary PIN (see FR-01).

- **AC:** Every worker screen has the Help tab reachable in 1 tap. Read-aloud gracefully hides itself when neither speech synthesis nor an audio clip is available.

8. Booking / Job State Machine



8.1 Transition Table

From	To	Actor	Notes
PENDING	ACCEPTED	Worker	Reveals phone numbers to both parties.
PENDING	DECLINED	Worker	Optional reason. Request returns to `OPEN`.
PENDING	CANCELLED	Customer	Reason required.
PENDING	EXPIRED	System	24 h without response. Counts against response score.
ACCEPTED	ON_THE_WAY	Worker	
ACCEPTED / ON_THE_WAY	CANCELLED	Customer or Worker	Reason required. Worker cancellations count against trust.
ON_THE_WAY	IN_PROGRESS	Worker	
IN_PROGRESS	AWAITING_CONFIRMATION	Worker	Optional notes / photos.
IN_PROGRESS	DISPUTED	Customer or Worker	
AWAITING_CONFIRMATION	COMPLETED	Customer	Creates verified work record.
AWAITING_CONFIRMATION	DISPUTED	Customer	Reason required.
AWAITING_CONFIRMATION	AUTO_CLOSED	System	72 h without response.
DISPUTED	COMPLETED / CANCELLED	Admin	Resolution note required.

Terminal states: `DECLINED`, `EXPIRED`, `CANCELLED`, `COMPLETED`, `AUTO_CLOSED`.

8.2 Implementation Rules

- All transitions go through one service function, e.g. `bookings.services.transition(booking, to_status, actor, note)`, which validates the transition table, checks the actor's permission, writes

`BookingStatusHistory`, emits notifications, and triggers trust recalculation — all inside `transaction.atomic()` with `select_for_update()` on the booking row.

- System transitions (`EXPIRED`, `AUTO_CLOSED`) run from a management command `python manage.py process_booking_timeouts`, scheduled every 15 minutes (cron on the host in MVP; Celery Beat later).

9. Trust Score Model

9.1 Factors and Weights

Weights come from the proposal and are configurable in settings (`TRUST_WEIGHTS`).

Factor	Weight	Normalized value (0–1)
Completion reliability	25%	`completed_or_auto_closed / accepted_bookings_in_terminal_state`
Customer rating	20%	`(bayesian_avg_rating - 1) / 4`
Verified jobs	15%	`min(verified_jobs / 20, 1)`
Cancellation behavior	15%	`1 - (worker_cancellations / accepted_bookings)`
Response time	10%	Median accept/decline time: ≤ 30 min = 1.0, ≥ 24 h = 0.0, linear between. Expired bookings count as 24 h.
Identity verification	10%	1 if identity `APPROVED` , else 0
Relevant experience	5%	`min(years_experience / 10, 1)`

```
trust_score = round(100 × Σ(weight × normalized_value))
```

9.2 Cold Start

- **Bayesian rating:** `bayesian_avg = (C × m + Σratings) / (C + n)` with `C = 5` , `m = 3.5` . Prevents one 5-star review from dominating.
- **Reliability, cancellation, response factors** use a neutral prior of 0.7 until the worker has 3 accepted bookings.
- Workers with fewer than **3 completed jobs** show a **"New on SkillPass"** label next to the score so customers know the data is thin.

9.3 Explainability

- Every calculation stores a JSON breakdown: per factor → raw value, normalized value, weight, points contributed.
- Public profile shows the breakdown, e.g.:

  > **Trust Score 82 / 100** Completion reliability — 24 / 25 (31 of 32 jobs completed) Customer rating — 17 / 20 (4.4 average from 28 reviews) Verified jobs — 15 / 15 (28 verified jobs) …

- `TrustScoreHistory` stores every recalculation (score, breakdown, trigger event, timestamp).

9.4 Rules

- Self-reported work never affects the score.
- Reviews marked invalid by admin are excluded.
- Disputed bookings resolved as `CANCELLED` due to worker fault count as worker cancellations; admin selects fault (`WORKER` , `CUSTOMER` , `NONE`) when resolving.

10. Matching Algorithm

Rule-based and deterministic in MVP.

10.1 Hard Filters (exclude if any fails)

- Worker is active, not suspended, availability ≠ `OFFLINE`.
- Worker has a skill in the requested category (and the requested skill, if specified).
- Worker serves the request's area.
- Worker has no active booking conflicting with the preferred date + time slot (P1).

10.2 Ranking Score

Component	Weight	Value
Skill match	30%	1.0 if exact skill match, 0.6 if category-only
Trust score	30%	`trust_score / 100`
Availability	15%	`AVAILABLE` = 1.0, `BUSY` = 0.4
Category experience	15%	`min(verified_jobs_in_category / 10, 1)`
Responsiveness	10%	Response-time factor from trust model

- Ties broken by most recent completed job.
- Return the top 20 with `match_score` and `match_reasons[]`.
- Weights configurable in settings (`MATCHING_WEIGHTS`).

11. Roles and Permissions

Action	Guest	Customer	Worker	Admin
Browse / search worker profiles	✓	✓	✓	✓
View public SkillPass profile	✓	✓	✓	✓
Create service request / booking	✗	✓	✗	✗
Accept / decline / progress booking	✗	✗	Own	✗
Confirm completion / dispute	✗	Own	✗	✗
Cancel booking	✗	Own	Own	✓
View booking details	✗	Own	Own	✓
Write review	✗	Own completed	✗	✗
Reply to review	✗	✗	Own (P1)	✗
Edit worker profile	✗	✗	Own	✓
Upload verification documents	✗	✗	Own	✗
View verification documents	✗	✗	Own	✓
Approve / reject verification	✗	✗	✗	✓
Report profile / review	✗	✓	✓	—
Resolve disputes, suspend users	✗	✗	✗	✓
Manage categories / skills / areas	✗	✗	✗	✓

"Own" = object-level check that the user is the booking's customer/worker or the profile owner.

12. Data Model

PostgreSQL. All tables have `id` (BigAutoField), `created_at`, `updated_at` unless noted.

12.1 Entity List

accounts

- `User` (custom, `AbstractBaseUser`): `phone` (unique, USERNAME_FIELD), `email` (nullable, unique), `full_name`, `role` (enum), `preferred_language` (`bn` / `en`), `is_active`, `is_suspended`, `is_staff`, `must_change_credential` (bool, set for admin-created accounts / temporary PINs), `failed_login_count`, `locked_at` (nullable), `created_by` (FK User, nullable — admin who did assisted onboarding). The password field stores the hashed password or hashed worker PIN.
- `CustomerProfile`: `user` (1-1), `default_area` (FK Area, nullable).

catalog

- `Division`, `District` (FK Division), `Area` (FK District): `name_bn`, `name_en`, `slug`.
- `ServiceCategory`: `name_bn`, `name_en`, `slug` (unique), `icon`, `is_active`, `sort_order`.
- `Skill`: `category` (FK), `name_bn`, `name_en`, `slug` (unique), `is_active`.

workers

- `WorkerProfile`: `user` (1-1), `display_name`, `photo`, `bio`, `years_experience`, `availability_status` (enum), `identity_verified` (bool, denormalized), `trust_score` (int, denormalized), `completed_jobs_count`, `verified_jobs_count`, `avg_rating` (decimal), `last_active_at`.
- `WorkerSkill`: `worker` (FK), `skill` (FK), `years_experience`, `evidence_verified` (bool). Unique (`worker`, `skill`).
- `WorkerServiceArea`: `worker`, `area`. Unique together.
- `WorkerAvailabilitySlot` (P1): `worker`, `day_of_week` (0–6), `slot` (`MORNING` / `AFTERNOON` / `EVENING`).
- `SelfReportedWork` (P1): `worker`, `category`, `title`, `description`, `approx_date`, photos via `WorkPhoto`.

bookings

- `ServiceRequest`: `customer` (FK User), `category`, `skills` (M2M Skill), `description`, `area`, `address_detail`, `preferred_date`, `preferred_slot`, `status`, `ai_suggested_category` (nullable), `ai_tags` (JSON, nullable).
- `Booking`: `service_request` (FK), `customer` (FK User), `worker` (FK WorkerProfile), `status` (enum), `accepted_at`, `completed_at`, `cancelled_by` (FK User, nullable), `cancel_reason`, `completion_notes`, `is_verified` (bool).
- Partial unique constraint: one active booking per `service_request`.
- `BookingStatusHistory`: `booking`, `from_status`, `to_status`, `actor` (FK User, nullable for system), `note`, `created_at`.
- `BookingPhoto` (P1): `booking`, `image`, `kind` (`REQUEST` / `COMPLETION`).

reviews

- `Review`: `booking` (1-1, unique), `customer`, `worker`, `quality`, `professionalism`, `punctuality`, `communication` (1–5 with `CheckConstraint`), `overall` (decimal, computed on save), `would_hire_again` (bool), `comment`, `is_hidden`, `is_invalid`.
- `ReviewReply` (P1): `review` (1-1), `text`.

verification

- `Verification`: `worker`, `type` (`IDENTITY` / `SKILL_EVIDENCE`), `worker_skill` (FK, nullable; required when type is `SKILL_EVIDENCE`), `status`, `reviewed_by` (FK User), `reviewed_at`, `rejection_reason`.
- `VerificationDocument`: `verification`, `file` (private storage), `original_name`, `content_type`, `size`.

trust

- `TrustScoreHistory`: `worker`, `score`, `breakdown` (JSON), `trigger` (string), `created_at`.

reports

- `Report`: `reporter`, `target_type` (`BOOKING` / `USER` / `REVIEW`), `target_id`, `reason` (enum), `description`, `status`, `assigned_admin`, `resolution_note`, `fault` (`WORKER` / `CUSTOMER` / `NONE`, nullable), `resolved_at`.

notifications

- `Notification`: `recipient`, `type` (enum), `title`, `body`, `data` (JSON: e.g. `booking_id`), `is_read`, `created_at`. Index on (`recipient`, `is_read`, `created_at`).

messaging (P1)

- `Message`: `booking`, `sender`, `text`, `created_at`, `read_at`.

audit

- `AuditLog`: `actor`, `action` (string), `target_type`, `target_id`, `metadata` (JSON), `ip_address`, `created_at`. Append-only.

organizations (P2)

- `Organization`, `OrganizationMember` (`organization`, `worker`, `role`, `joined_at`, `left_at`).

Payments: not modeled in MVP (out of scope).

12.2 Key Integrity Rules

- A `Review` requires its booking's status to be `COMPLETED` or `AUTO_CLOSED` and `review.customer == booking.customer` (service-layer validation + DB unique on `booking`).
- A `Booking` worker must have a skill in the request's category at creation time.
- `CheckConstraint`s for rating ranges, non-negative experience, and non-negative counters.
- Denormalized counters on `WorkerProfile` are updated only through service functions, never directly in views.

13. API Specification

- Base path: `/api/v1/`. JSON only. Auth via `Authorization: Bearer <access_token>`.
- Pagination: page-number, `?page=`, `page_size` default 20, max 100.
- Errors: `{ "detail": "...", "code": "...", "errors": { "field": ["msg"] } }`.
- OpenAPI schema at `/api/schema/`, Swagger UI at `/api/docs/` (drf-spectacular).

13.1 Auth — `/api/v1/auth/`

Method	Path	Description	Access
POST	`register/`	Create account (customer: `password`; worker: 6-digit `pin`)	Public
POST	`login/`	Obtain access + refresh tokens (`phone` + `password`, where a worker sends the PIN)	Public
POST	`refresh/`	Rotate refresh token	Public
POST	`logout/`	Blacklist refresh token	Auth
GET / PATCH	`me/`	Current user	Auth
POST	`password/change/`	Change password or PIN (also used for forced first-login change)	Auth
POST	`password/reset/` + `password/reset/confirm/`	Email reset (P1)	Public

13.2 Catalog

Method	Path	Description	Access
GET	`categories/`	List active categories	Public
GET	`categories/{slug}/skills/`	Skills in a category	Public
GET	`skills/`	List / filter skills	Public
GET	`areas/`	List areas (filter by district)	Public
POST / PATCH / DELETE	`categories/`, `skills/`, `areas/`	Manage catalog	Admin

13.3 Workers — `/api/v1/workers/`

Method	Path	Description	Access
GET	`/`	Search: `?category=&skill=&area=&min_trust=&verified=&available=&ordering=`	Public
GET	`{id}/`	Public SkillPass profile	Public
GET	`{id}/work-history/`	Verified + self-reported history	Public
GET	`{id}/reviews/`	Reviews (paginated)	Public

Method	Path	Description	Access
GET	`{id}/trust-score/`	Score + breakdown	Public
GET / PATCH	`me/`	Own profile	Worker
PATCH	`me/availability/`	Set availability status	Worker
PUT	`me/skills/`	Replace skill set	Worker
PUT	`me/service-areas/`	Replace service areas	Worker
GET / POST / DELETE	`me/self-reported-work/`	Manage self-reported work (P1)	Worker
GET	`me/dashboard/`	Today's jobs, pending requests, stats	Worker

13.4 Service Requests — `/api/v1/service-requests/`

Method	Path	Description	Access
POST	`/`	Create request	Customer
GET	`/`	Own requests	Customer
GET / PATCH	`{id}/`	Detail / edit while `OPEN`	Owner
POST	`{id}/cancel/`	Cancel request	Owner
GET	`{id}/matches/`	Ranked matching workers	Owner
POST	`classify/`	AI category suggestion from free text (P2)	Customer

13.5 Bookings — `/api/v1/bookings/`

Method	Path	Description	Access
POST	`/`	Create booking `{service_request, worker}`	Customer
GET	`/`	Own bookings (`?status=&role=`)	Customer / Worker
GET	`{id}/`	Detail incl. status timeline	Participant / Admin
POST	`{id}/accept/`	PENDING → ACCEPTED	Worker
POST	`{id}/decline/`	PENDING → DECLINED	Worker
POST	`{id}/on-the-way/`	ACCEPTED → ON_THE_WAY	Worker
POST	`{id}/start/`	ON_THE_WAY → IN_PROGRESS	Worker
POST	`{id}/mark-complete/`	IN_PROGRESS → AWAITING_CONFIRMATION	Worker
POST	`{id}/confirm/`	AWAITING_CONFIRMATION → COMPLETED	Customer
POST	`{id}/cancel/`	→ CANCELLED (reason required)	Participant
POST	`{id}/dispute/`	→ DISPUTED (reason required)	Participant

Method	Path	Description	Access
GET / POST	`{id}/messages/`	Booking message thread (P1)	Participant

> **Design decision:** explicit action endpoints (`/accept/` , `/start/` , …) instead of a generic `PATCH /status/` . Each action maps to one button in the UI, has its own permission check, and keeps the state machine easy to test.

13.6 Reviews — `/api/v1/reviews/`

Method	Path	Description	Access
POST	`/`	Create review for a booking	Customer (eligible)
POST	`{id}/reply/`	Worker reply (P1)	Worker (owner)

13.7 Verification — `/api/v1/verification/`

Method	Path	Description	Access
POST	`/`	Submit verification (multipart: type, worker_skill?, files[])	Worker
GET	`/`	Own verifications / admin queue (`?status=PENDING`)	Worker / Admin
GET	`documents/{id}/`	Stream private document	Owner / Admin
POST	`{id}/approve/`	Approve	Admin
POST	`{id}/reject/`	Reject `{reason}`	Admin

13.8 Reports, Notifications, Admin

Method	Path	Description	Access
POST	`reports/`	Create report	Auth
GET	`reports/`	Admin queue / own reports	Admin / Auth
POST	`reports/{id}/resolve/`	Resolve `{resolution_note, fault, booking_outcome?}`	Admin
GET	`notifications/`	List (`?unread=true`)	Auth
GET	`notifications/unread-count/`	Badge count	Auth
POST	`notifications/{id}/read/` , `notifications/read-all/`	Mark read	Auth
GET	`admin/stats/`	Dashboard metrics	Admin
GET / PATCH	`admin/users/` , `admin/users/{id}/`	Search users, suspend / reactivate, unlock locked account	Admin
POST	`admin/users/{id}/reset-credential/`	Issue a temporary PIN / password (sets `must_change_credential`)	Admin

Method	Path	Description	Access
POST	`admin/workers/`	Assisted onboarding: create worker account with temporary PIN (P1)	Admin

13.9 Example: Worker Profile Response

```
{
  "id": 42,
  "display_name": "Rahim Uddin",
  "photo": "https://.../rahim.jpg",
  "years_experience": 9,
  "availability_status": "AVAILABLE",
  "badges": {
    "identity_verified": true,
    "verified_jobs": 28,
    "new_on_skillpass": false
  },
  "trust_score": {
    "score": 82,
    "breakdown": [
      { "factor": "completion_reliability", "points": 24, "max": 25, "detail": "31 of 32 jobs completed" },
      { "factor": "customer_rating", "points": 17, "max": 20, "detail": "4.4 average from 28 reviews" }
    ]
  },
  "skills": [
    { "slug": "ac-gas-refill", "name": "AC Gas Refill", "evidence_verified": true }
  ],
  "service_areas": [{ "slug": "mirpur", "name": "Mirpur" }],
  "rating": { "average": 4.4, "count": 28, "would_hire_again_pct": 93 },
  "member_since": "2026-03-01"
}
```

14. UI / UX Requirements

14.1 Principles

- **Worker UI is the simplest UI.** Most workers have limited formal education and little smartphone experience (see persona Rahim). A worker should be able to use SkillPass by looking at pictures and pressing big buttons, with little reading and almost no typing.
- **Bengali first.** Default language for workers is Bengali; customers can switch Bengali / English anytime. All UI strings go through `react-i18next`; no hard-coded text.
- **Mobile first.** Designed at 360 px width first, then scaled up.
- **Evidence first.** Profiles show verified facts before self-descriptions.

14.2 Worker UI Rules

Principle	Rule
Everyday Bengali	Short, spoken-style Bengali words ("কাজ শেষ"), never technical terms ("Mark as Completed"). Max ~6 words per label.
Pictures over words	Every category, skill, action, and status has an icon. Choices are picture tiles, not text lists.
Color means status	Green = new / available, yellow = in progress, blue = done, red = cancelled — always with an icon and a word, never color alone.
One screen, one job	Each screen asks for one thing and has one big primary button, fixed at the bottom of the screen.
Big touch targets	Buttons ≥ 56 px tall and full width; tap targets ≥ 48 × 48 px; worker body text ≥ 18 px.
Almost no typing	Tap tiles, toggles, number steppers, and pickers. Phone and PIN fields open the number keypad (`inputmode="numeric"`). Free text only for optional fields, with voice typing (P1).
Voice help	🔊 read-aloud button on each worker screen (P1, FR-22).
Same layout everywhere	Bottom bar with only 4 icons: **হোম (Home), কাজ (Jobs), প্রোফাইল (Profile), সাহায্য (Help)**. Primary action always in the same place.
Safe mistakes	Destructive actions (Decline, Cancel) are small secondary buttons and ask for confirmation with an icon dialog.
Plain errors	Short Bengali message that says what to do next. No codes or technical text.
Stay logged in	Long session (30-day refresh token) so workers rarely see the login screen.

Job flow buttons: গ্রহণ করুন (Accept) → রওনা দিয়েছি (On the Way) → কাজ শুরু (Start) → কাজ শেষ (Complete). Only the next step is shown, as one large button.

14.3 Key Worker Screens

Worker home (top to bottom):

1. Greeting with the worker's name + 🔊 button.
2. One large availability switch: 🟢 "আমি কাজের জন্য প্রস্তুত" (I'm ready for work) / 🟣 "এখন ব্যস্ত" (busy now).
3. New job requests as large green cards: category icon, area, time, and two buttons — ✓ গ্রহণ (Accept) and ✗ না (No).
4. Today's jobs as cards with a colored status tag.

5. Trust score card: ⭐ score and ✓ number of verified jobs, in big numbers.
6. Bottom bar: Home, Jobs, Profile, Help.

Active job screen (top to bottom):

1. Category icon + job title, customer first name, area.
2. Two large buttons side by side: 📞 **কল করুন** (Call) and 🗺️ **রাস্তা দেখুন** (Directions).
3. Step tracker with 4 dots: Accepted ✓ · On the way ✓ · Started · Completed.
4. One large primary button at the bottom for the next step.
5. Small "Report a problem" link.

Help screen:

1. Big 📞 **Call support** and **WhatsApp support** buttons.
2. Tutorial video (P1).
3. 5 common questions as picture cards with short answers and 🔊 read-aloud.

14.4 Screen Inventory

Public

- Landing page (search bar: category + area)
- Worker search results
- Worker SkillPass profile
- Trust score explainer
- Login (phone + password / PIN) / Register (picture-card role picker)

Customer

- Dashboard (active bookings, recent requests)
- New service request form
- Match results for a request
- Booking detail (status timeline, worker contact once accepted, confirm / dispute actions)
- Review form
- Notifications

Worker

- Home: availability switch, new requests, today's jobs, trust score card
- Booking request detail (accept / decline)
- Active job screen (one big next-step button, call, directions)
- Profile wizard (picture tiles, camera) / edit profile
- Verification upload (camera)
- My work history (+ add self-reported work, P1)
- My reviews
- Notifications
- Help (call / WhatsApp support, FAQ cards, tutorial video)
- Set new PIN (forced on first login after assisted onboarding or PIN reset)

Admin

- Dashboard (metrics)
- Verification queue + document viewer

- Disputes / reports queue + resolution form
- Users (search, suspend, unlock, reset PIN / password)
- Assisted worker registration (P1)
- Catalog management (categories, skills, areas)

15. Non-Functional Requirements

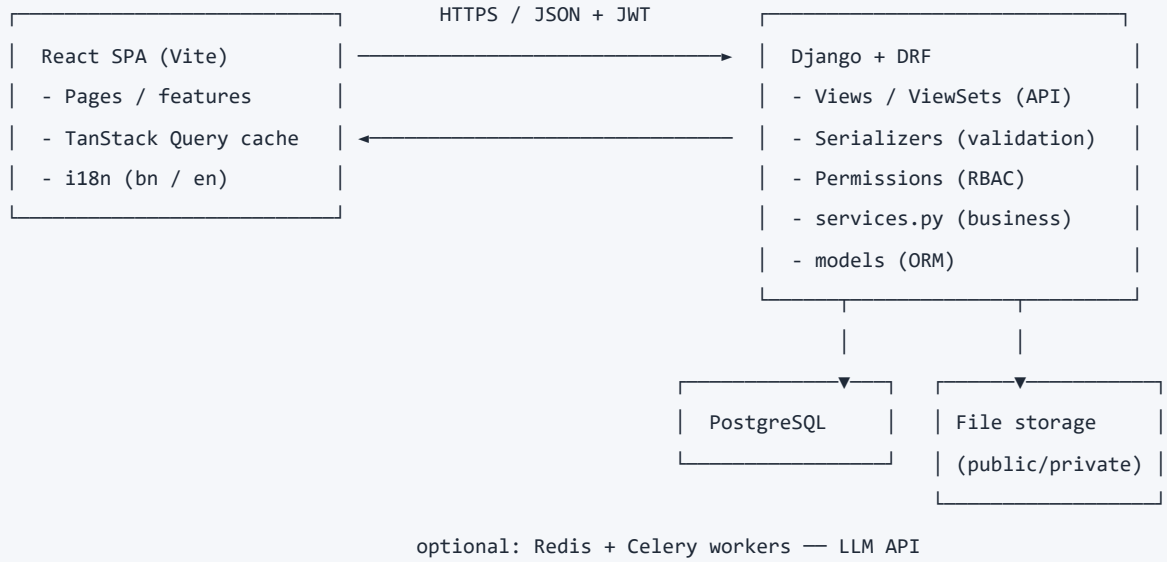
Area	Requirement
Security	Passwords and worker PINs hashed with Django's default (PBKDF2 or Argon2); PIN accounts protected by throttling + lockout (FR-01). JWT with short-lived access tokens and refresh rotation + blacklist. Every endpoint has an explicit permission class. Object-level checks on all participant resources. CORS restricted to the frontend origin. Upload validation (type by content, size limit). Verification documents in private storage. Secrets via environment variables only. DRF throttling on auth and write endpoints.
Privacy	Phone shown only to booking participants after acceptance. Full address shown only to the assigned worker. Public work history shows area-level location only. Store the minimum personal data needed.
Performance	p95 API latency < 500 ms for list/detail endpoints with seed data (≈ 500 workers, 5,000 bookings). Avoid N+1 queries via `select_related` / `prefetch_related`. Denormalized counters for search sorting. DB indexes on filter columns.
Low-end devices	Worker pages must work well on a budget Android phone (2 GB RAM) over slow mobile data: route-level code splitting, initial JS for worker routes < 250 KB gzipped, compressed/resized images, no heavy animations. Worker home usable within 3 s on Chrome DevTools "Fast 3G" + 4× CPU slowdown.
Reliability	All state transitions are atomic and row-locked. Status history is append-only. Timeouts processed idempotently.
Usability	A first-time worker completes the profile wizard in < 5 minutes. From the worker home screen, accepting a job takes 1 tap and finishing it takes 3 more taps. In usability testing, at least 4 of 5 workers with limited literacy complete "accept and finish a job" without help.
Accessibility	WCAG 2.1 AA contrast. Semantic HTML, labels on all inputs, keyboard navigable. Worker UI: tap targets ≥ 48 px, primary buttons ≥ 56 px tall, body text ≥ 18 px, every icon paired with a text label. Bengali font with proper rendering (e.g., Hind Siliguri / Noto Sans Bengali).
Maintainability	Modular Django apps; business logic in `services.py`, not views or serializers. One frontend API layer (`src/api` + `src/hooks`); pages never call the API directly. Linting/formatting enforced (ruff + black; ESLint + Prettier). Backend test coverage ≥ 70%.
Scalability	Architecture allows adding Redis caching, Celery tasks, and WebSockets later without rewriting core logic.
Auditability	Verification decisions, dispute resolutions, suspensions, and booking transitions are recorded with actor and timestamp.
Browser support	Latest Chrome (Android + desktop), Firefox, Edge, Safari.

16. Technical Architecture

16.1 Stack

Layer	Technology
Frontend	React 19 + Vite 8 (plain JavaScript / JSX), React Router 7, TanStack Query 5 (server state), Axios (with JWT refresh interceptor), Tailwind CSS 4, React Hook Form + Zod 4 (forms/validation), react-i18next (bn/en), lucide-react (icons), Recharts (admin charts)
Worker assistance	Web Speech API (speech synthesis for read-aloud, speech recognition for voice typing, P1), Vibration API (new-job alert), vite-plugin-pwa (installable app, P1)
Backend	Python 3.12, Django 5.2 LTS, Django REST Framework 3.18
Auth	djangorestframework-simplejwt (rotation + blacklist app)
Database	PostgreSQL 16+ (psycopg 3); developed on PostgreSQL 18
API docs	drf-spectacular (OpenAPI 3 + Swagger UI)
Filtering	django-filter
Other backend libs	django-cors-headers, Pillow, python-decouple / django-environ
Testing	pytest, pytest-django, factory_boy, pytest-cov (backend); Vitest + React Testing Library (frontend); Playwright for 1–2 E2E flows (P2)
Code quality	ruff, black (backend); ESLint, Prettier (frontend); enforced in CI
Version control / CI	Git + GitHub, GitHub Actions (lint + test on PR)
Local dev	Local PostgreSQL install; frontend can run alone on mock data (`VITE_USE_MOCKS=true`)
Deployment	Backend + DB on Render / Railway; frontend on Vercel / Netlify; media on cloud object storage (P1)
Optional infra	Redis + Celery (+ Celery Beat) for timeouts, notifications, AI calls (P2)
AI (optional)	LLM API behind an `AIProvider` interface (P2)

16.2 Architecture



Layering rule: views are thin. They parse input with serializers, call a service function, and serialize output. All business rules (transitions, trust calculation, matching, review eligibility) live in each app's `services.py` and are unit-tested directly.

16.3 Repository Structure

Member-owned folders are marked. Everything else is owned by the project lead.

```
skillpass-bd/
├── backend/                          # Lead
│   ├── config/                       # settings (base/dev/prod), urls, wsgi/asgi
│   ├── apps/
│   │   ├── accounts/                 # User, PIN/password auth, profiles
│   │   ├── catalog/                  # categories, skills, areas
│   │   ├── workers/                  # WorkerProfile, skills, areas, availability
│   │   ├── bookings/                 # ServiceRequest, Booking, state machine, matching
│   │   ├── reviews/
│   │   ├── verification/
│   │   ├── trust/                    # trust calculation + history
│   │   ├── reports/                  # reports & disputes
│   │   ├── notifications/
│   │   ├── audit/
│   │   └── ai/                       # P2, AIProvider + keyword fallback
│   ├── common/                       # shared permissions, pagination, exceptions
│   ├── fixtures/                     # seed categories, skills, areas
│   └── requirements/                 # base.txt, dev.txt, prod.txt
├── frontend/
│   └── src/
│       ├── app/                      # Lead: main.jsx, App.jsx, router.jsx, providers
│       ├── api/                      # Lead: axios client, endpoint functions
│       ├── hooks/                    # Lead: data hooks (mock <-> real API switch)
│       ├── mocks/                    # Lead: mock JSON data
│       ├── lib/                      # Lead: formatters, constants, speech helpers
│       ├── layouts/                  # Lead: Public / Customer / Worker / Admin layouts
│       ├── components/
│       │   ├── ui/                   # Lead: Button, Input, Modal, Table ...
│       │   └── domain/               # Lead: WorkerCard, TrustBreakdown, ReviewCard,
│       │                             #       StatusTag, StatusTimeline, StarRating
│       ├── i18n/                     # translations (bn / en)
│       │   ├── index.js, en|bn/common.json     # Lead
│       │   ├── en|bn/public.json               # M2
│       │   ├── en|bn/customer.json             # M3
│       │   ├── en|bn/worker.json               # M4
│       │   └── en|bn/admin.json                # M5
│       └── features/                 # each: pages/, components/ (routes live in app/router.jsx)
│           ├── _example/             # Lead: reference pages
│           ├── public/               # M2
│           ├── customer/             # M3
│           ├── worker/               # M4
│           └── admin/                # M5
├── docs/                             # PRD, tasks, handbooks, guides, diagrams, testing
├── scripts/                          # Lead: ownership.json, sync_lead.py, make_zips.py
└── .github/                          # CI, PR template, CODEOWNERS
```

16.4 Configuration (Environment Variables)

`DJANGO_SECRET_KEY`, `DJANGO_DEBUG`, `DJANGO_ALLOWED_HOSTS`, `DATABASE_URL`, `CORS_ALLOWED_ORIGINS`, `JWT_ACCESS_MINUTES`, `JWT_REFRESH_DAYS`, `JWT_WORKER_REFRESH_DAYS`, `SUPPORT_PHONE`, `SUPPORT_WHATSAPP`, `MEDIA_STORAGE_*`, `EMAIL_*` (P1), `AI_ENABLED`, `AI_PROVIDER`, `AI_API_KEY` (P2). Frontend: `VITE_API_BASE_URL`.

17. Release Plan and Milestones

The project is built **part by part**. There is no fixed deadline per part; each part is finished, pushed, and merged before the next starts.

Team model: the project lead builds the database, backend, shared frontend foundation, and deployment. Four members (M2–M5) each own one frontend area (pages, area components, translations) and related documentation. Member code is delivered as per-part zips and pushed by each member through their own pull requests; the lead reviews and merges every PR.

Part	Scope	Pushed by
1. Foundation	Repo, Django + Vite scaffolds, settings, CI, UI kit, layouts, router, mocks, scripts, stub pages	Lead
2. Auth	User model, PIN/password auth, JWT, roles / login, register, set-PIN, navbar, footer	Lead → M2
3. Catalog	Categories, skills, areas, seed data / catalog admin page	Lead → M5
4. Worker profiles & search	Profiles, search API / landing, search, public profile, worker wizard, worker home	Lead → M2, M4
5. Bookings	Requests, matching, state machine, timeouts / customer booking pages, worker job pages	Lead → M3, M4
6. Reviews & trust	Reviews, work history, trust score / review form, reviews & history pages, trust explainer	Lead → M2, M3, M4
7. Verification	Uploads, private documents, approval / upload page, queue, document viewer	Lead → M5
8. Reports & disputes	Reports, resolution, suspension, PIN reset / report modal, disputes queue, users page	Lead → M2, M5
9. Notifications & help	Notifications, new-job alert / bell, notifications page, account settings, worker Help screen	Lead → M3, M4
10. Admin dashboard	Stats API / dashboard page	Lead → M5
11. Worker assistance (P1)	Read-aloud, voice typing, PWA, tutorial video, assisted onboarding	Lead → M4, M5
12. Final	Testing, usability test with workers, fixes, deployment, docs, demo	Lead + all
Stretch	AI classification + profile assistant, contractor module, Celery, messaging	Lead

17.1 MVP Definition of Done (P0)

1. Authentication and role management
2. Worker profiles, skills, categories
3. Service areas and availability status
4. Customer service requests
5. Worker search and rule-based matching
6. Booking and full job lifecycle
7. Completion confirmation
8. Verified work history

9. Structured reviews
10. Worker verification (identity + skill evidence)
11. Explainable trust score
12. In-app notifications, worker new-job alert
13. Admin dashboard, verification queue, disputes, moderation
14. Worker-friendly UI: PIN login, picture-tile choices, one-button job flow, call / directions buttons, Help screen

18. Success Metrics

18.1 Project (Academic) Metrics

Metric	Target
P0 requirements implemented	100%
Backend test coverage	≥ 70% (100% of booking transitions and trust calculation)
End-to-end demo flow	Register → profile → request → match → book → complete → confirm → review → score update, with no manual DB edits
API documentation	Every endpoint in Swagger with request/response schemas
Security tests	Authorization tests for every participant-scoped endpoint
Usability test	5 participants including real technicians with limited literacy; at least 4 of 5 complete "accept and finish a job" unaided

18.2 Product Metrics (If Piloted)

Metric	Definition
Profile completion rate	% of registered workers with ≥ 80% profile completeness
Request → booking conversion	% of service requests that get an accepted booking
Booking completion rate	% of accepted bookings ending `COMPLETED`
Confirmation rate	% of `AWAITING_CONFIRMATION` bookings confirmed (vs auto-closed)
Review rate	% of completed bookings with a review
Worker response time	Median time from booking creation to accept/decline
Verification turnaround	Median time from submission to admin decision

19. Risks and Mitigations

Risk	Impact	Mitigation
Low worker literacy / little smartphone experience	Workers abandon onboarding or cannot finish jobs in the app	PIN login, picture tiles, one-button job flow, big controls, almost no typing, read-aloud and voice typing, Help screen with call/WhatsApp support, assisted onboarding by admin, Bengali tutorial video, usability testing with real technicians
Forgotten PIN	Worker locked out	Admin/support issues a temporary PIN; long-lived session so login is rare
Weak PIN guessing	Account takeover	Weak-PIN blocklist, rate limiting, account lock after 10 failures in 24 h
Fake profiles	Customer trust drops	Separate verification badges, admin review, reporting, suspension
Fake reviews	Trust score manipulation	Reviews only on eligible completed bookings, one per booking, Bayesian averaging, admin invalidation
Collusion (fake bookings to inflate score)	Inflated verified-job counts	Flag repeated bookings between the same pair (P1), cap counted verified jobs from one customer, admin audit
Sensitive document exposure	Privacy breach	Private storage, authenticated streaming, strict permission tests, minimal retention
Scope creep	Core workflow not finished	Strict P0/P1/P2 discipline; AI, contractor, payments, GPS are stretch only
Insufficient data for matching/AI	Weak recommendations	Deterministic rule-based matching; AI is optional with keyword fallback
Bengali AI quality	Wrong categories suggested	AI only suggests; user always confirms; keyword fallback
Team coordination	Integration delays	API contract first (OpenAPI), feature branches + PR reviews, CI on every PR

20. Open Questions

#	Question	Proposed default
Q1	Should customers be able to see a worker's price or rate?	No pricing in MVP; price agreed directly. Add an optional "visit charge" field in P1.
Q2	Phone verification by SMS OTP at registration?	Not in MVP (cost). Revisit if an SMS gateway is available for the demo.
Q3	Which areas beyond Dhaka city to seed?	Dhaka city only for MVP.
Q4	Should workers be allowed to decline without penalty?	Declines do not hurt the score; expiries do (response factor).
Q5	Can a customer review after `AUTO_CLOSED`?	Yes, within 14 days of auto-close.
Q6	Which LLM provider for the AI module?	Decide at S6 based on Bengali quality and cost; code against `AIProvider`.
Q7	Is the contractor module required for grading?	Treat as P2 unless the supervisor requires it.
Q8	Where do media files live in production?	Local disk in dev; cloud object storage (e.g., S3-compatible) in production.
Q9	Is Bengali speech (read-aloud / voice typing) available on target phones?	Test on 2–3 budget Android phones early. Where Bengali voices are missing, use pre-recorded audio clips for read-aloud and hide voice typing.
Q10	Who answers the support phone / WhatsApp during the demo?	A team member's number, configured via `SUPPORT_PHONE` / `SUPPORT_WHATSAPP`.
Q11	Should customers also be allowed to use a PIN?	No for MVP; PIN is for workers only. Revisit after usability testing.

End of document.